Access Control Semantic Caching

Laurent d'Orazio, Univ Rennes, CNRS, IRISA

2025/2026

1 Introduction

It is widely recognized that some of the data collected, processed and stored in current information systems is of a personal nature, making it sensitive and requiring appropriate protection. Organizations, whether public or private, e.g., in domains like healthcare, finance, logistics that handle such data are consequently subject to increasingly stringent regulations. It is then crucial to equip information systems with mechanisms that control and restrict access to sensitive data to authorized users only. While experts in cybersecurity are involved in the design of advanced mechanisms to ensure data protection, an increasing number of individuals and organizations with limited technical expertise are tasked with managing large volumes of sensitive data. In most cases, these non-expert users are actually more aware of the operational realities of their domain and often have a more accurate and detailed vision of the field. However, the design and implementation of security policies in the actual information systems remain complex tasks that typically require significant expertise in formal security models. This complexity creates a gap between non-expert stakeholders, who are able to express their security requirements in natural language or through informal specifications, and the technical mechanisms needed to enforce these requirements effectively. As a result, there is a risk that the intended security objectives are either misinterpreted or only partially implemented, leading to vulnerabilities or inconsistencies.

The convergence of cloud computing [1] and Big Data technologies have attracted increasing attention during the last decades [2] [3] [4] [5] [6] [7] . According to the market research organization MarketsandMarkets[1], the global big data market was estimated to be valued at $162.6 billion in revenue in 2021 and is projected to reach $273.4 billion by 2026, growing at a CAGR of 11.0% from 2021 to 2026 [x]. In particular, in-memory data store/cache market is projected to grow from $8 billion in 2024 to $30 billion by 2033[2].

[1] https://www.marketsandmarkets.com

[2] https://www.imarcgroup.com/inmemory- database-market.

2 Background

2.1 LLM

Large Language Models (LLMs) [14] are sophisticated neural architectures developed using huge collections of text data to forecast and produce coherent, contextually appropriate natural language. They generally use transformer architectures [15], where self-attention mechanisms allow the model to identify long-range dependencies in text. Notable instances include GPT (Generative Pre-trained Transformer) [16] and its subsequent versions (e.g., LlaMA [17]), which vary in parameter quantity, training data, and tuning approaches. LLMs are used in a wide variety of linguistic and reasoning tasks, including summarization, question answering, translation, and verbalization of structured data, which makes them ideal for creating narratives about museum objects from formal ontological data.

Effective utilization of LLMs requires prompt engineering [18], a process of designing LLM inputs (a.k.a., prompts - e.g., questions, instructions, context, and examples) that guide the model's behavior toward a desired output (specific content, tone, or structure). In this context, a large number of prompting methodologies have been proposed, and they keep evolving rapidly, introducing sophisticated strategies to enhance controllability and task alignment of LLM-generated outputs. These methodologies range from simple instruction-based expressions (e.g., zero-shot prompting [19]) to multi-stage, reasoning-oriented (e.g., chain-of-thought prompting [20]), and retrieval-augmented generation (RAG) [21] approaches, reflecting the growing need to balance creativity with consistency and domain particularity.

Apart from text generation, LLMs rely on and produce dense vector representations of linguistic elements, called embeddings. An embedding maps distinct elements (e.g., words, sentences, or documents) into points within a continuous, low-dimensional vector space so that geometric relationships (e.g., cosine similarity, Euclidean distance) represent semantic similarity [22]. Early work on embeddings focused on word embeddings [23], i.e., vector representations for individual word types.

Embeddings are learned from large text corpora, and provide a reusable interface of the raw text. Transformer-based LLMs produce contextual token (i.e., a single segment of text that the model perceives as a fundamental symbol) embeddings, where each token's representation depends on the entire surrounding sequence through self-attention [24]. For many applications, however, it is desirable to represent entire sentences, paragraphs, or short documents as single vectors. Sentence embeddings (or, document embeddings) [25, 26] address this need by mapping variable-length text spans into fixed-dimensional vectors that are directly comparable via standard similarity measures.

2.2 Agentic AI

Multi-agents Michael J. Wooldridge: An Introduction to MultiAgent Systems, Second Edition. Wiley 2009, ISBN 978-0-470-51946-2, pp. I-XXII, 1-461

Auto-GPT[3] Use case in digital twins [8]

2.3 Semantic Caching and Semantic Caching

Semantic caching [23], [10], [27], [21], [9], initially introduced in distributed data bases, allows to exploit resources in the cache and knowledge contained in the queries themselves. It enables effective reasoning (analysis and processing), delegating part of the computation process to the cache, reducing both data transfers and CPU load on servers. Nevertheless, reasoning on the cache content induces non-negligible overhead [20]. This thus raises the challenge on providing finely-tuned caches with appropriate analysis and processing capability with respect to access patterns (for instance supervisors with first coarse grain queries and then refining requests), performances indicators (response time, quality/freshness, energy consumption, etc.) and hardware acceleration opportunities.

More recently semantic caching has been referred, yet differently, in LLMs to store LLM responses. User queries are first sent to the cache for a response before being sent to LLMs. If the cache has the answer to a query, it returns the answer to the user without having to query the LLM. This approach saves costs on API recalls and makes response times faster. Moreover, network fluctuations will not affect the response time of the cache, making it stable.

3 AC-GPT

3.1 Overview

Figure 1 provides an overview of the global proposal, called AC-GPT (Access Control-Generative Pre-trained Transformer). In this paper we focus on the real-time security monitoring module. This module consists of three main components that will be detailed bellow: (1) a module to convert suspicious behaviors into codified templates of suspicious behaviors; semantic caching layers finely tuned with respect to the templates for both LLM inference and massively parallel security monitoring and (3) hardware acceleration to improve performances not only in response time but also in energy consumption.

3.2 SATISFIES

In access control systems, converting suspicious behaviors into codified templates (or rules/patterns) helps automate threat detection by defining clear, actionable criteria for anomalies. SATISFIES, that stands for Suspicious beHAviors inTo codIfied templateS oF suspIcious bEhaviorS, as its name suggested

[3] https://agpt.co

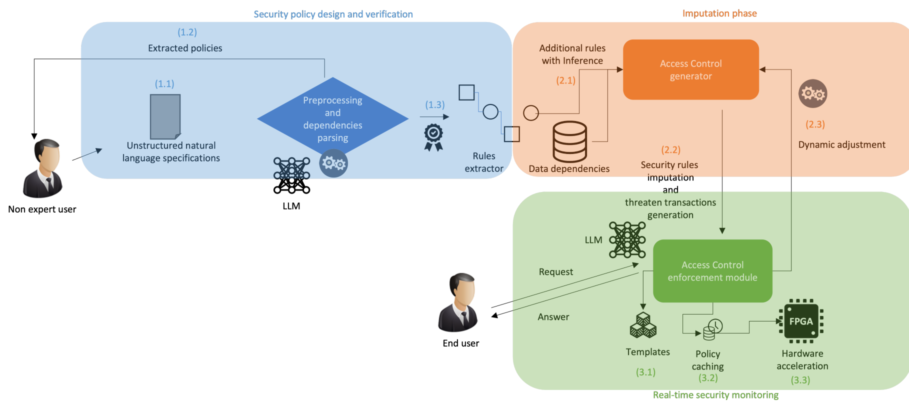

Figure 1: Overview

aims to convert suspicious behaviors into codified templates of suspicious behaviors.

3.2.1 Raw suspicious behaviors

Below is an example of how raw suspicious behaviors, that can be structured into codified templates using a combination of (1) time-based, (2) spatial, (3) credential-based and (4) behavioral attributes, on corporate building AC. The objective is to detect potential tailgating, credential abuse or insider threats in a secured facility with RFID badges and CCTV.

- Tailgating:
  - . Two people enter a secure door with only one badge swipe.
  - . The door remains open longer than the standard 3–second threshold.
- Credential Sharing: The same badge is used to unlock two different doors in distant locations within an impossible timeframe (e.g., 5 seconds).
- Unusual Access Times: An employee badges into a restricted lab at 3 AM when their usual hours are 9 AM 5 PM.
- Failed Access Attempts: Five failed badge swipes in 10 seconds at a high–security door, followed by a successful entry.
- Lingering in Sensitive Areas: A contractor spends 30 minutes in a server room when the average visit duration is 5 minutes.
- Device Spoofing: A badge ID appears in the system logs but wasnt issued to any employee (cloned/

```
    counterfeit).
- Unusual Path Patterns: An employee moves from the lobby
     to the CEOs floor without passing through
    intermediate checkpoints.
```

3.2.2 Codified templates

The system will convert the above raw suspicious behaviors into codified templates, rules and patterns, as if-then rules or JSON/YAML templates for an access control system (e.g., SIEM, IAM, or custom analytics engine). Using the previous running examples, the templates bellow consist is illustrations.


```
{
  "rule_id": "AC-001",
  "name": "Tailgating_Detected",
  "description": "Two entries detected with a single
     badge swipe.",
  "conditions": {
    "trigger": "door_unlocked",
    "time_window": 5,  // seconds
    "events": [
      {
        "type": "badge_swipe",
        "badge_id": "BADGE_123",
        "door_id": "DOOR_A",
        "timestamp": "T0"
      },
      {
        "type": "motion_sensor",
        "door_id": "DOOR_A",
        "timestamp": "T0 + 1s",
        "person_count": 2  // >1 person detected
      }
    ]
  },
  "actions": [
    "alert_security",
    "lock_door",
    "flag_badge_for_review"
  ],
  "severity": "high"
}

{
  "rule_id": "AC-002",
  "name": "Credential_Sharing",
```

```
  "description": "Same badge used in two distant
     locations in an impossible timeframe.",
  "conditions": {
    "badge_id": "BADGE_456",
    "events": [
      {
        "type": "access_granted",
        "door_id": "DOOR_B",
        "location": "Floor_1",
        "timestamp": "T0"
      },
      {
        "type": "access_granted",
        "door_id": "DOOR_C",
        "location": "Floor_10",
        "timestamp": "T0 + 5s",  // < min travel time (e.
           g., 60s)
        "max_allowed_travel_time": 60
      }
    ]
  },
  "actions": [
    "revoke_badge_temporarily",
    "notify_manager",
    "trigger_investigation"
  ],
  "severity": "critical"
}

{
  "rule_id": "AC-003",
  "name": "OffHours_Access",
  "description": "Access attempt outside of employee's
     typical working hours.",
  "conditions": {
    "badge_id": "BADGE_789",
    "employee_id": "EMP_001",
    "typical_hours": ["09:00-17:00", "Mon-Fri"],
    "current_time": "03:00",
    "day_of_week": "Tuesday",
    "door_id": "DOOR_D",
    "restricted_area": true
  },
  "actions": [
    "request_justification",
    "escalate_if_no_response"
```

```
  ],
  "severity": "medium"
}

{
  "rule_id": "AC-004",
  "name": "BruteForce_Badge",
  "description": "Multiple failed swipes followed by
     success (possible stolen/cloned badge).",
  "conditions": {
    "door_id": "DOOR_E",
    "failed_attempts": 5,
    "time_window": 10,  // seconds
    "followed_by": {
      "type": "access_granted",
      "timestamp": "T0 + 11s"
    }
  },
  "actions": [
    "lock_badge",
    "dispatch_guard_to_door",
    "review_cctv_footage"
  ],
  "severity": "high"
}

{
  "rule_id": "AC-005",
  "name": "Extended_Stay_Alert",
  "description": "Person remains in a restricted area
     beyond expected duration.",
  "conditions": {
    "badge_id": "BADGE_101",
    "area_type": "server_room",
    "entry_time": "T0",
    "exit_time": null,
    "current_time": "T0 + 30m",  // > avg duration (5m)
    "role": "contractor"  // Non-IT role
  },
  "actions": [
    "send_notification_to_person",
    "escalate_if_no_exit_in_5m"
  ],
  "severity": "medium"
}
```

3.2.3 Tuning

The proposed solutions may consider different data sources like badge swipe logs (timestamp, badge ID, door ID), motion sensors/weight sensors (for tailgating), CCTV metadata (for path analysis) or HR systems (for typical working hours/roles).

Thresholds need to be taken into account, like adjust time windows (e.g., max_allowed_travel_time) based on facility layout or define "unusual" behavior statistically (e.g., 3from average dwell time).

3.2.4 Integration

AC-GPT aimes to feed templates into access control software or into a SIEM (like Splunk or IBM QRadar). Future work may include to use Machine Learning to refine templates over time, which is out of the scope of this report.

3.2.5 Workflow

3.3 Semantic Caching Layers

3.4 MASCARA++

4 Related Work

4.1 Key Value stores and Cache-stores

Key-Value (KV) stores (also referred as key-value databases), represent a data storage paradigm designed for storing, retrieving, and managing associative arrays, a data structure more commonly known today as a dictionary or hash table. Representative research is the domain are RocksDB [9], FASTER [10], SplinterDB [11], LeanStore [12], LevelDB[4] or Memcached[5].

Cache-stores, can be defined as a storage server that exposes one or more remote like Redis[6], Valkey[7], KeyDB[8] or Dragonfly[9] endpoints that adhere to a well-defined wire protocol and a rich (or expressive) command set []. Cloud providers offer services around these cache-stores like Google Memory Store[10], Amazon MemoryDB[11], Amazon ElasticCache[12] or Microsoft Azure Redis Cache[13].

[4]https://github.com/google/leveldb
[5]https://memcached.org
[6]https://github.com/google/leveldb
[7]https://github.com/google/leveldb
[8]https://github.com/google/leveldb
[9]https://github.com/google/leveldb
[10]https://github.com/google/leveldb
[11]https://github.com/google/leveldb
[12]https://github.com/google/leveldb
[13]https://github.com/google/leveldb

While KV stores and even more cache-stores can be seen as a possible storage layer for LLM or data semantic caching, they do not consider the specific requirements, in particular in terms of semantic, for these use cases.

LLM in DB Panel at VLDB [13]

Cents: a unified and cost-effective framework for LLM-based solutions for table understanding tasks. https://www.vldb.org/pvldb/vol18/p4574-xiao.pdf

References

[1] M. Armbrust, A. Fox, R. Griffith, A. D. Joseph, R. H. Katz, A. Konwinski, G. Lee, D. A. Patterson, A. Rabkin, I. Stoica, and M. Zaharia, "A view of cloud computing," *Communications of the ACM*, vol. 53, no. 4, pp. 50–58, 2010.

[2] D. Agrawal, S. Das, and A. E. Abbadi, "Big data and cloud computing: New wine or just new bottles?" in *Proc. VLDB Endow.*, 2010, pp. 1–12.

[3] J. Cohen, B. Dolan, M. Dunlap, J. M. Hellerstein, and C. Welton, "Mad skills: New analysis practices for big data," in *Proc. VLDB Endow.*, 2009, pp. 1–12.

[4] C. Bizer, P. Boncz, M. L. Brodie, and O. Erling, "The meaningful use of big data: four perspectives - four challenges," in *SIGMOD Rec.*, 2011, pp. 1–12.

[5] S. Chaudhuri, "How different is big data?" in *International Conference on Data Engineering (ICDE)*, A. Kementsietsidis and M. A. V. Salles, Eds., 2012.

[6] D. Agrawal, S. Das, and A. E. Abbadi, "Big data and cloud computing: current state and future opportunities," in *EDBT 2011, 14th International Conference on Extending Database Technology, Uppsala, Sweden, March 21-24, 2011, Proceedings*, A. Ailamaki, S. Amer-Yahia, J. M. Patel, T. Risch, P. Senellart, and J. Stoyanovich, Eds. ACM, 2011, pp. 530–533. [Online]. Available: https://doi.org/10.1145/1951365.1951432

[7] V. R. Borkar, M. J. Carey, and C. Li, "Inside "big data management": ogres, onions, or parfaits?" in *15th International Conference on Extending Database Technology, EDBT '12, Berlin, Germany, March 27-30, 2012, Proceedings*, E. A. Rundensteiner, V. Markl, I. Manolescu, S. Amer-Yahia, F. Naumann, and I. Ari, Eds. ACM, 2012, pp. 3–14. [Online]. Available: https://doi.org/10.1145/2247596.2247598

[8] A. Timms, A. Langbridge, A. Antonopoulos, A. Mygiakis, E. Voulgari, and F. O'Donncha, "Agentic ai for digital twin." in *AAAI*, 2025.

[9] Z. Cao, S. Dong, S. Vemuri, and D. H. C. Du, "Characterizing, modeling, and benchmarking rocksdb key-value workloads at facebook." in *FAST*, 2020.

[10] B. Chandramouli, G. Prasaad, D. Kossmann, J. J. Levandoski, J. Hunter, and M. Barnett, "Faster: A concurrent key-value store with in-place updates." in *SIGMOD Conference*, 2018.

[11] A. Conway, A. Gupta, V. Chidambaram, M. Farach-Colton, R. P. Spillane, A. Tai, and R. Johnson, "Splinterdb: Closing the bandwidth gap for nvme key-value stores." in *USENIX ATC*, 2020.

[12] V. Leis, M. Haubenschild, A. Kemper, and T. Neumann, "Leanstore: Inmemory data management beyond main memory." in *ICDE*, 2018.

[13] A. Y. Halevy, Y. Choi, A. Floratou, M. J. Franklin, N. F. Noy, and H. Wang, "Will llms reshape, supercharge, or kill data science?" *Proc. VLDB Endow.*, vol. 16, no. 12, 2023.